

REV : Caching, Offloading, or Structural Locality? A Head-to-Head Study of DEX and DART on RDMA Disaggregated Memory

Abstract

Disaggregated memory (DM) splits a server into a compute pool of CPU-rich, memory-poor nodes and a memory pool of CPU-poor, memory-rich nodes connected by RDMA. Two recent ordered indexes attack the resulting remote-access bottleneck from opposite directions. *DART* is a lock-free two-layer hashed Adaptive Radix Tree that minimizes remote round trips *structurally*—an Express Skip Table and adaptive hashed nodes keep every operation to a handful of one-sided reads while the memory node does *zero* CPU work. *DEX* is a path-aware B^+ -tree that instead spends two *tunable* resources: a compute-side directory *cache* and optional RPC *offloading* that runs index logic on memory-node CPUs to ship results instead of pages. We build a 1-to-1 harness that runs both systems on the same RDMA cluster, working set, thread budget, and latency instrumentation, and ask a single question: *when, and by what mechanism, does a tunable caching/offloading index catch a structurally local one?* Across point lookups and range scans under uniform and Zipfian-0.99 access, we sweep cache (64–512 MB), the offload switch, and the number of memory-node service threads (`memThreadCount`). We find that (i) pure caching alone rarely catches DART—it pays a full page read on every miss; (ii) offloading is the mechanism that lets DEX overtake DART, collapsing per-operation RDMA reads from 2.3–22 to ≈ 0 by shipping an 8 B value instead of a 0.5–1 KB page; (iii) the crossover is workload-shaped: under Zipfian lookups DEX leads at *every* cache size, while uniform range scans need the largest cache and still trail in tail latency; and (iv) offloading moves a measurable slice of useful work onto memory-node CPUs, where extra service threads buy throughput only until the network or compute node becomes the bottleneck. We frame these as a portrait of three operating points and discuss when each design is the right one.

Keywords

Disaggregated memory, RDMA, ordered index, caching, compute offloading, Adaptive Radix Tree, B^+ -tree

1 Introduction

Disaggregated memory (DM) architectures decouple computing and memory into separate pools joined by a high-speed interconnect such as RDMA or CXL [1–3]. Compute nodes hold powerful CPUs but little local memory; memory nodes hold large capacity but few or no usable cycles. The central design tension is well known: a one-sided RDMA operation costs $\sim 2\mu\text{s}$ —an order of magnitude more than local memory—and RDMA bandwidth (100–200 Gbps) is far below local memory bandwidth. A DM index must therefore minimize *both* the number of round trips and the bytes moved per operation, while remaining scalable and crash-consistent under many concurrent clients.

Two recent systems answer this challenge with fundamentally different philosophies, and this paper places them side by side.

DART: minimize round trips structurally. DART [1] is a lock-free two-layer hashed ART. A hash-based *Express Skip Table* provides express paths that skip the upper levels of the radix tree, halving traversal depth; an *Adaptive Hashed Layout* confines each in-node search to one small hash bucket instead of a whole node; and *Decoupled Metadata Organization* embeds node metadata in the parent pointer so writes become a single failure-atomic 8 B compare-and-swap. The result is an index whose remote cost is low *by construction*: operations take a handful of one-sided reads, and—crucially—the memory node executes *no* CPU work at all. Every access is served by the NIC; the memory-node process blocks on a socket for the duration of the run. DART’s performance is therefore largely *insensitive to compute-side cache*.

DEX: spend cache and memory-node CPU. DEX is a path-aware B^+ -tree for DM. It keeps a compute-side directory *cache* of inner nodes, and—optionally—*offloads* the leaf access as an RPC executed on a memory-node service thread, returning the 8 B value rather than shipping a 0.5–1 KB page back to the client. DEX thus exposes two tunable knobs that DART lacks by design: *how much cache* and *whether to offload*. Offloading deliberately migrates a slice of useful work onto the memory node, trading remote CPU cycles for network bytes.

The question. These are not just two indexes; they are two *positions* on how to spend the DM resource budget. DART refuses to spend memory-node CPU and leans on structural locality. DEX is willing to spend both cache and remote CPU to shrink the bytes on the wire. The natural question—and the one practitioners actually face when choosing or provisioning a DM index—is:

When does a tunable caching/offloading index catch a structurally local one, by what mechanism, and at what cost in memory-node CPU?

This paper. We answer empirically with a controlled 1-to-1 comparison (Section 3). We equalize the working set (50 M 16 B entries), the thread budget (32 compute + 4 memory threads), and the latency artifact (a shared 500 ns-bucket histogram yielding p99 on *both* systems), and we sweep three axes that map exactly onto the design knobs:

- **Cache** (64, 128, 256, 512 MB) — the caching axis (Section 4).
- **Offload** (off / on) — the result-shipping axis (Section 5).
- **memThreadCount** (2, 4, 6) — the remote-thread axis (Section 6).

all crossed with operation $\in \{\text{point lookup, range scan}\}$ and distribution $\in \{\text{uniform, Zipfian-0.99}\}$.

Contributions and findings.

- (1) A *fair-comparison harness* for two DM indexes with opposite philosophies, including a per-node CPU sampler and a memory-node *active-work* tracker that distinguishes “cores pegged by busy-poll” from “useful work done” (Section 7).
- (2) **Caching alone rarely catches DART.** DEX with offloading *off* pays a full page read on every miss; its per-op RDMA reads fall only slowly with cache and it never overtakes DART on uniform workloads (Section 4).
- (3) **Offloading is the catch-up mechanism.** Turning offload on collapses per-op RDMA reads from 2.3–22 down to ≈ 0 , and DEX overtakes DART: by 1.7 $\times$ on uniform lookups and $\approx 3\times$ on Zipfian lookups at 512 MB (Section 5).
- (4) **The crossover is workload-shaped.** Under Zipfian lookups DEX leads at *every* cache size; uniform range scans need the most cache and still trail DART in tail latency until ~ 2 GB (extrapolated) (Sections 4–5).
- (5) **Remote threads have a knee.** With offload on, adding memory-node service threads raises throughput only while the memory node is the bottleneck; past ~ 4 threads the gain flattens as the network or compute node saturates (Section 6).

2 Background and the Two Designs

2.1 Disaggregated memory and RDMA

A DM compute node accesses the memory pool with one-sided RDMA READ/ WRITE (bypassing the remote CPU) and one-sided atomics (RDMA CAS/FAA) that update a single 8 B word. The cost model has two terms that an index must jointly minimize: *round trips* (each $\sim 2\mu\text{s}$, and serialized by node-to-node dependencies during a traversal) and *bandwidth* (read amplification, e.g. fetching a whole node to find one pointer). A third axis—*where the cycles run*—is usually ignored because classic one-sided designs leave the memory-node CPU idle; it becomes central once a system offloads work (Section 7).

2.2 DART: structural locality, idle memory node

DART minimizes the two cost terms in the structure itself. The Express Skip Table is a DM-friendly extendible hash table whose directory (<1 MB) is cached at the compute node, so an express node is reached in a *single* remote read that skips 2^k levels of the tree. Within an inner node, the Adaptive Hashed Layout hashes the next partial key to one 128 B bucket, so an in-node search moves one bucket rather than the whole node. Decoupled Metadata Organization folds node type/length into the 8 B compound child pointer, giving lock-free, failure-atomic writes via a single RDMA CAS and removing the extra round trip that header metadata would cost. The defining systems property for this study: DART’s memory node performs *no* CPU work—it sets up queue pairs, signals ready, and blocks on a socket until the run ends. All of DART’s useful work lives on the compute node, and its only “remote cost” is network round trips, not remote cycles.

2.3 DEX: a path-aware cache plus RPC offloading

DEX is a B^+ -tree whose compute nodes hold a *directory cache* of inner (path) nodes. A lookup walks the cached path and, on reaching the leaf level, either (a) *reads the leaf page* over one-sided RDMA (offload *off*), or (b) *issues an RPC* to a memory-node service (directory) thread that performs the leaf access locally and returns just the value (offload *on*). Mode (a) is a pure caching design directly comparable to DART—no memory-node CPU—but it pays a full page read on every leaf miss. Mode (b) trades a memory -node CPU cycle for a large reduction in bytes on the wire: instead of a 0.5–1 KB page, the network carries an 8 B value. DEX thus has two knobs DART structurally lacks: cache size and the offload switch. The number of memory-node service threads, `memThreadCount`, bounds how much offloaded work the memory node can absorb in parallel.

2.4 Why a busy-poll CPU number is misleading

Both systems busy-poll the NIC: DEX directory threads run `while(true){pollWait` on pinned cores, and compute workers spin on completion queues. Raw CPU% is therefore pinned near 100% whether or not real work is happening. To measure “remote load” meaningfully we instrument the *active-work fraction*—time spent inside `process_message` divided by directory-thread wall time—alongside a per-node `process/system` CPU sampler. This lets us separate *cores pegged* by the poll from *useful work done*, which is exactly the distinction that makes offloading visible (Section 7).

3 Experimental Methodology

We run both systems on a 2-node RDMA cluster (Linux, MLNX_OFED): one compute node and one memory node. To make the comparison fair we equalize everything that is not the design under test.

Working set and matrix. Both sweeps load 50 M keys (16 B entries) and measure 30–50 M operations.

op	$\in$ {point lookup, range scan (100 keys)}
dist	$\in$ {uniform, Zipfian-0.99}
cache	$\in$ {64, 128, 256, 512} MB (compute-side)

DEX names map onto DART’s: `range=scan`, `zipfian=zipf99`.

Thread budget. We match **32 compute + 4 memory** threads. DART has no memory-side service threads (its memory node is idle by construction), so “4 memory threads” is a DEX-only knob that we additionally sweep in Section 6. DART runs 32 compute threads; its per-thread cache budget is recomputed so the *total* cache matches DEX at each point.

Three system curves. Because DART has no offload axis, we run DEX in two modes and carry three curves on every cache plot: **DART**, **DEX offload-off** (pure caching, the apples-to-apples baseline against DART), and **DEX offload-on** (`MANUAL_PUSHDOWN`, RPC enabled).

Latency artifact. We made the two systems emit an identical latency histogram—ops per 500 ns bucket plus per-op `p50/p90/p99/max`—so latency is compared **p99-to-p99** rather than DEX-p99 vs. DART-average. All latency and load instrumentation compiles away when

Table 1: Throughput (Mops/s) at the cache endpoints (64 MB → 512 MB). DART is flat (single value). Bold marks the best system at 512 MB.

op	dist	DART	DEX off	DEX on
lookup	uniform	2.9	1.29 → 2.52	2.17 → 4.93
lookup	zipf	2.9	2.04 → 5.06	3.30 → 8.66
scan	uniform	1.29	0.42 → 0.56	0.90 → 1.46
scan	zipf	1.30	0.60 → 0.95	1.29 → 2.37

disabled; its overhead is one clock pair per op, negligible against an RDMA round trip.

Plots. All figures are produced by a single script over the committed sweep CSVs, so every number in the text is reproducible from the same source. Throughput is in Mops/s (higher is better); latency is p99 in μ s (lower is better).

4 The Caching Axis: Does More Cache Catch DART?

We first vary cache with offload held in each state. Figure 1 shows throughput; Figure 2 shows p99 latency. The DART curve is essentially *flat*: from 64 to 512 MB its lookup throughput holds near 2.8–2.9 Mops and scan near 1.25–1.30 Mops, with p99 pinned at $\approx 20.5 \mu$ s (lookup) and $\approx 40.5 \mu$ s (scan). This flatness is the structural-locality story: DART’s skip table keeps the hot path resident with a <1 MB directory, so a larger compute-side cache buys it almost nothing.

Pure caching (offload-off) rarely catches DART. DEX offload-off pays a full page read on every leaf miss. Its per-op RDMA reads fall only slowly with cache—for uniform lookups from 6.79 down to 3.34 over the $8\times$ cache range—and throughput rises correspondingly slowly: from 1.29 to 2.52 Mops, *still below* DART’s ≈ 2.9 at the largest cache. On uniform scans it crawls from 0.42 to 0.56 Mops against DART’s 1.29—it is nowhere close. The lone exception is Zipfian, where caching a small hot set is genuinely effective: DEX offload-off lookups reach 5.06 Mops at 512 MB, *passing* DART’s 2.9 once cache ≥ 256 MB. Even so, Zipfian *scans* top out at 0.95 Mops, below DART’s 1.30. Table 1 summarizes.

Tail latency. Figure 2 (p99 vs. cache) and Figure 4 (the focused tail view with crossover extrapolation) tell the same story for the SLO-relevant number. DART’s p99 is a flat horizontal line; DEX-off’s p99 falls with cache but stays above DART on uniform workloads. The tail view fits DEX’s falling p99 against $\log_2(\text{cache})$ and extrapolates to where it would meet DART’s flat tail: for uniform scans that crossover is ~ 2068 MB and for Zipfian scans ~ 1284 MB—i.e. well beyond the swept range, confirming that for scans DART’s tail is hard to beat with cache alone.

5 The Offloading Axis: Result-Shipping vs. Page-Shipping

Caching changes *how often* DEX touches the network; offloading changes *how much it moves when it does*. This is the mechanism that lets DEX overtake DART.

RDMA reads collapse. With offload on, the dominant leaf miss becomes an RPC that returns an 8 B value instead of a page. Per-op RDMA reads collapse accordingly. For uniform lookups they fall from 2.33 at 64 MB to 0.0008 at 512 MB—essentially zero remote reads per operation once the inner path is cached and every leaf access is an RPC. Zipfian lookups behave the same ($1.30 \rightarrow 0.001$). The work has not disappeared; it has *moved*: *rpc/op* sits near 1.0 for lookups (one leaf RPC per op) and 0.95–1.37 for scans (a scan touches several leaves, which offload collapses into a small number of RPCs plus one scratch read).

DEX overtakes DART. The byte reduction translates directly into throughput (Figure 1, red curve). DEX offload-on overtakes DART:

- **Lookup / uniform:** crosses DART between 64 and 128 MB; reaches 4.93 Mops at 512 MB, $\approx 1.7\times$ DART.
- **Lookup / Zipfian:** already above DART at 64 MB (3.30 vs. 2.9) and leads at *every* cache size, reaching 8.66 Mops, $\approx 3\times$ DART—free hot-set hits compound with result-shipping.
- **Scan / uniform:** the hardest case; it catches DART only near 512 MB (1.46 vs. 1.29), because each scan offloads several leaf reads.
- **Scan / Zipfian:** passes DART by ~ 128 MB and reaches 2.37 Mops, $\approx 1.8\times$ DART at 512 MB.

Where DEX catches DART, at a glance. Figure 3 plots the DEX/DART p99 *ratio* against cache; below the dashed 1.0 line DEX’s tail is faster. The offload-on curve dives under 1.0 for lookups within the swept range (DEX-on lookup p99 reaches 14.0μ s uniform / 12.5μ s Zipfian at 512 MB, inside DART’s $\approx 20 \mu$ s operating range), while offload-off stays above 1.0 on uniform workloads—a compact statement of “offloading, not caching, is what closes the tail gap.”

The “how” in one line. DEX catches DART by *converting page-shipping into result-shipping* (offload) while keeping the *inner path resident* (cache $\geq$ the inner working set). It catches *earlier under skew* (free hot-set hits) and *needs more cache for scans* (more leaves per op). DART wins precisely when you cannot or should not spend memory-node CPU: it stays flat-fast with zero remote load, most visibly on uniform scans.

6 The Remote-Thread Axis: memThreadCount

Offloading only helps if the memory node can serve the RPCs. Each directory thread serves RPCs serially while it busy-polls, so the number of service threads, `memThreadCount`, bounds offloaded throughput. We sweep it over {2, 4, 6} with offload off and on across the full matrix. Figure 5 plots throughput vs. `memThreadCount` at the largest cache; Figure 6 plots throughput vs. cache with one curve per thread count.

Offload-off is the control—and it is flat. With offload off the memory node does no CPU work, so throughput is independent of `memThreadCount`: uniform lookups read 1.17–1.18 Mops at 64 MB whether 2, 4, or 6 threads are configured. This is the guardrail: any movement in the offload-off curve would indicate a confound, and there is none.

Offload-on has a knee. With offload on, adding service threads adds parallel memory-node capacity, so offloaded ops queue less. At 256 MB, uniform lookups rise from 2.98 Mops (2 threads) to 4.46

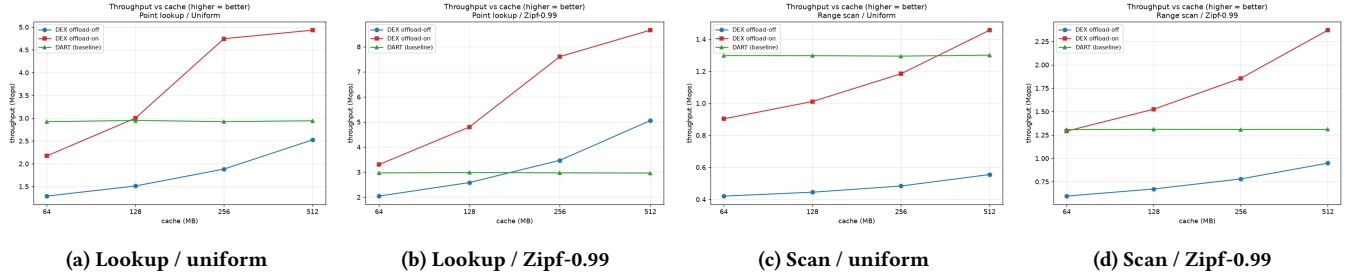


Figure 1: Throughput vs. cache. DART (green) is flat—insensitive to compute-side cache. DEX offload-off (blue) rises slowly and, on uniform workloads, never catches DART. DEX offload-on (red) climbs steeply and overtakes DART once the inner path fits in cache; it leads at *every* cache size for Zipfian lookups.

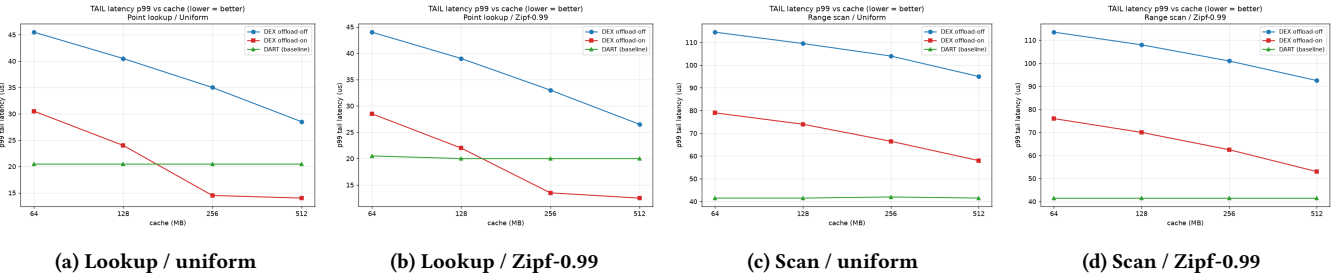


Figure 2: p99 tail latency vs. cache (lower is better). DART (green) is flat; DEX offload-on (red) drops below DART’s tail for lookups within the swept range, while for scans it approaches but does not cross it before 512 MB.

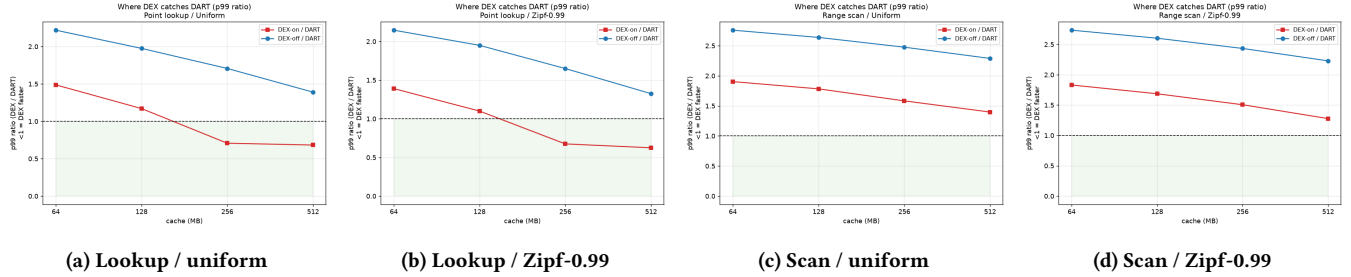


Figure 3: Where DEX catches DART: p99 ratio DEX/DART vs. cache. Shaded region (< 1.0) is where DEX’s tail is faster than DART’s. Offload-on (red) crosses into the region for lookups; offload-off (blue) largely does not on uniform workloads.

(4) to 4.73 (6); at 512 MB from 3.33 to 4.62 to 4.81. The gain is steep from 2 to 4 threads and then *flattens*: past ~ 4 threads the bottleneck has moved off the memory node—onto the network (NIC message rate) or the compute node (it cannot issue RPCs faster)—so extra threads buy little. That inflection is the practical answer to “how many memory -node threads should I provision for offloading”: about four for this workload, beyond which the budget is better spent elsewhere.

Interaction with cache. Figure 6 shows the two knobs are not independent. At small cache the memory node is busiest (many leaf RPCs), so more threads help most; at large cache the inner path is resident and the limit shifts to the compute/network side, so the

thread curves bunch together. The two DEX knobs are complementary: cache decides how *often* you offload, threads decide how fast the memory node *absorbs* what you offload.

7 Where the Cycles Live: Remote vs. Compute Load

The three operating points differ not only in throughput but in *where the useful work runs*—the axis disaggregation is supposed to care about.

DART and DEX-caching: memory node idle. DART’s memory node does zero useful work by construction, and DEX offload-off is the same—caching is one-sided, so the memory-node CPU is idle (busy-polling but doing nothing). All useful work is on the

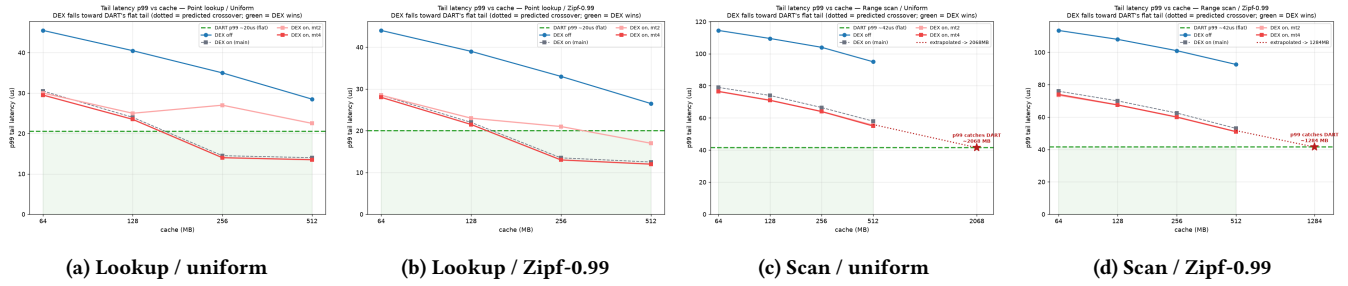


Figure 4: Focused tail-latency view. DART’s flat p99 (green dashed) is the target; DEX offload-on (reds, one curve per memThreadCount) falls toward it as cache grows. For scans the dotted line extrapolates the DEX-on fit to the cache where it would meet DART (~2068 MB uniform, ~1284 MB Zipfian).

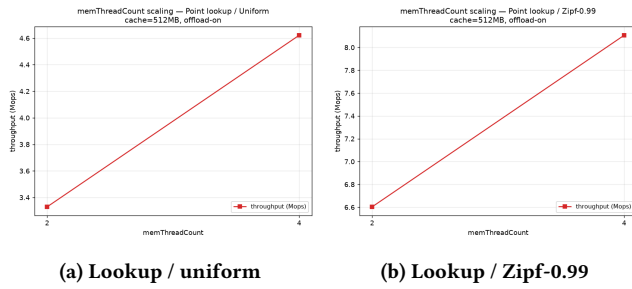


Figure 5: Throughput vs. memThreadCount (offload-on, largest cache). Throughput rises while the memory node is the bottleneck, then flattens past ~4 threads.

Table 2: Where the cycles live. “Useful” is the active-work fraction; both systems busy-poll, so gross CPU% is pinned regardless.

node / mode	gross CPU%	useful work
DART memory	≈0 (blocked)	0 (passive RDMA target)
DART compute	high (workers+spin)	all index logic
DEX memory, off	≈100%/thread (spin)	≈0 active
DEX memory, on	≈100%/thread (spin)	>0, rises with rpc
DEX compute	high (workers+spin)	index + RPC issue

compute node. This is “pure” disaggregation: the memory pool is dumb memory.

DEX-offload: a measured slice migrates. Offload deliberately moves useful work onto the memory node. Even though gross CPU% was already pinned by the poll, the *active-work fraction* rises above zero and grows with the RPC rate—this is the only regime where memory-node CPU load, and hence memThreadCount, matters at all. The $\text{rpc/op} \approx 1$ (lookups) to 1.37 (uniform scans) figures from Section 5 quantify exactly how much work migrates per operation. Table 2 summarizes the four regimes. The takeaway: DEX buys its throughput lead by spending a resource—memory-node cycles—that DART refuses to touch.

8 Synthesis: Three Operating Points

Pulling the axes together, each system occupies a distinct position in a five-dimensional trade-off space: *Throughput*, *Tail speed* ($1/\text{p99}$), *Network thrift* ($1/\text{RDMA-reads-per-op}$), *Cache robustness* (flatness across the cache sweep), and *Disaggregation purity* (how little memory-node CPU it uses—maximal for DART and DEX-off, reduced for DEX-on).

- **DART — the spiky specialist.** Strong on *network thrift*, *cache robustness*, and *disaggregation purity*; respectable on *tail speed*; weak only where it structurally cannot compete—it can never trade memory-node CPU for network bytes. Its advantages stand out most under Zipfian-small-cache.
- **DEX offload-off — the same shape, smaller.** It shares DART’s “no memory-node CPU” profile but with worse network thrift (full page reads), so it sits inside DART on most axes.
- **DEX offload-on — the balanced, larger envelope.** It activates the latent offload capability, pushes network thrift to the maximum ($\text{reads/op} \rightarrow 0$), and takes the throughput/tail lead at adequate cache—enclosing the other two under uniform/large-cache—paying for it only in disaggregation purity.

A decision rule. Choose DART when memory-node CPU is unavailable or must stay idle (strict disaggregation, shared memory pools, untrusted/limited memory nodes), when cache is scarce, or for scan-heavy uniform workloads where its flat tail is hard to beat. Choose DEX with offloading when memory-node cycles are spendable and the inner path fits in cache—especially under skew, where it leads everywhere, or when network bandwidth is the binding constraint and shipping 8 B beats shipping a page.

9 Discussion and Threats to Validity

Absolute throughput and thread count. Throughput numbers are absolute and depend on the matched 32/4 thread budget; the *shapes* (flat DART, slow DEX-off, steep DEX-on, the knee in memThreadCount) are the robust findings. DART’s cache flatness and DEX’s RDMA-read collapse are mechanism-level effects that do not depend on the exact thread count.

Crossover precision. The scan tail crossovers (~2068 MB uniform, ~1284 MB Zipfian) are log-linear extrapolations beyond the swept

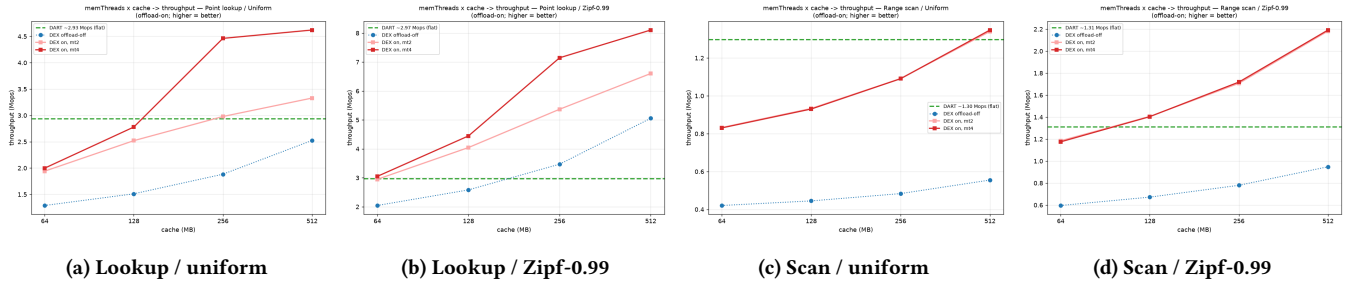


Figure 6: Joint $\text{memThreadCount} \times \text{cache}$ effect on DEX offload-on throughput, with DEX offload-off and DART’s flat baseline for reference. More threads lift the whole curve while the memory node is the bottleneck; the lift shrinks once cache makes the compute/network side the limit.

range and should be read as “DART’s scan tail is not caught within practical cache,” not as exact cache budgets.

Remote-load magnitudes. We report the *direction* and the per-op RPC counts that drive memory-node load; absolute active-work percentages depend on the reporter window and are used qualitatively (Table 2).

Single memory node. The topology is one compute / one memory node. Multi-node scaling, failure recovery, and write-heavy/mixed workloads are natural extensions; both systems provide the crash-consistency primitives (DART’s failure-atomic CAS, DEX’s RPC path) to support them.

10 Related Work

DM indexing has produced both tree and hash designs. SMART [2] is the state-of-the-art DM-based ART and caches inner nodes at the compute node, but is limited by compute-side cache capacity and a FIFO cache policy, and reads whole nodes (read amplification). Sherman [3] is a write-optimized DM B⁺-tree using lock-based concurrency with hierarchical locks. DART [1] removes both the round-trip and bandwidth costs structurally and adds lock-free, failure-atomic writes. RACE hashing [4] is the DM-friendly extendible hash that DART’s Express Skip Table builds on. DEX represents the caching-plus-offloading lineage, where compute pushes work into the memory pool to save network bytes. Our contribution is not a new index but a *controlled head-to-head* that isolates

the mechanisms—caching, offloading, remote threads—and shows exactly which one closes the gap and at what cost.

11 Conclusion

DART and DEX embody opposite answers to the disaggregated-memory index question. DART minimizes remote cost *structurally* and keeps the memory node idle; DEX spends *cache* and *memory-node CPU* to do the same. Our 1-to-1 study shows that caching alone rarely catches DART, that *offloading* is the mechanism that lets DEX overtake it—collapsing per-op RDMA reads to near zero and leading by up to $\approx 3\times$ under skew—and that the crossover is workload-shaped, with uniform scans the hardest case. The advantage is bought with memory-node cycles whose absorption is gated by a small number of service threads. The practical lesson is a portrait of three operating points rather than a single winner: structural locality when the memory node must stay dumb, caching-plus-offloading when its cycles are yours to spend.

References

- [1] B. Zhang et al. DART: A Lock-free Two-layer Hashed ART Index for Disaggregated Memory. *Proc. ACM Manag. Data* 4(1) (SIGMOD), 2026.
- [2] X. Luo et al. SMART: A High-Performance Adaptive Radix Tree for Disaggregated Memory. *OSDI*, 2023.
- [3] Q. Wang et al. Sherman: A Write-Optimized Distributed B+Tree Index on Disaggregated Memory. *SIGMOD*, 2022.
- [4] P. Zuo et al. One-sided RDMA-Conscious Extendible Hashing for Disaggregated Memory (RACE). *USENIX ATC*, 2021.